RDFIA 1 - ab

1. Train set : learn the parameters of the model by minimizing the training loss.

   Validation set : tune hyperparameters (learning rate, regularization, etc.) and detect overfitting.

   Test set : evaluate the final performance of the model on unseen data.

2. A larger N improves generalization and reduces overfitting.

   A smaller N may cause overfitting.

3. Without activation functions, successive linear transformation are equivalent to a single linear transformation, so the network could not model non-linear relationships. Activation functions introduce non-linearity, so that network can learn complexe decision boundaries.

4. n_x : the size of the input

   n_h : the size of the hidden layer (number of hidden neurons)

   n_y : the size of the output

   In practice, n_x and n_y are fixed by the data, while n_y is a hyperparameter chosen by the practitioner.

5. $\hat{y}$: the predicted output of the network

   y : the ground-truth label, represented in one-hot coding.

   The difference is that $\hat{y}$ is the model's prediction, while y is the true target.

6. SoftMax converts raw output scores into prabability that sum to 1, which makes interpretation easier and is suitable for classification tasks where we want to choose the most likely class.

Scanné avec CamScanner

7. Starting from input x:

$$\tilde{h} = W_h x + b_h$$
$$h = \tanh(\tilde{h})$$
$$\tilde{y} = W_y h + b_y$$
$$\hat{y} = \text{SoftMax}(\tilde{y})$$

8. • For cross-entropy: $\hat{y}_i$ must get closer to the true class probability (the component of y equal to 1 should approach 1, and all others should approach to 0).

• For squared error: $\hat{y}_i$ must get closer to the true value y_i.

9. Cross-entropy is better for classification because its output represents a probability distribution and measures the dissimilarity between predicted and true distributions. Mean squared error is better for regression because it penalizes the squared difference between continuous predictions and continuous ~~ff~~ targets.

10.

	Advantages	Disadvantages
Classic (full batch)	stable convergence, exact gradient	very slow for large datasets, high memory cost
Mini-batch SGD	good balance between effiency and stability	slight noise, need to choose batch size
Online	fast updates, adopt quickyly	very noisy convergence, may oscillate

Mini-batch SGD is the most reasonable because it balances speed, noise and stability.

11. A small η means slow convergence and may get stuck in local minima. A large η means faster learning but might overshoot the minimum or diverge. Therefore, choosing η is crucial, we often tune it empirically or adapt it dynamically.

12. Naive approach: Redundant computations, exponential or at least quadratic in the number of layers ($O(n^2)$ or worse)

Backpropagation: Reuses intermediate gradients efficiencly, linear complexity in the number of layers ($O(n)$)

Thus, backpropagation is more efficient.

Scanné avec CamScanner

13. The network must be differentiable (each layer's operation has a well-defined derivative.

The architecture should be continuous and composed of differenciable functions. (affine + activation).

Parameters must be connected to the loss through the computational graph.

14. $\hat{y}_i = \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}}$

$l = -\sum_{i=1}^{n_y} y_i \log \hat{y}_i = -\sum_{i=1}^{n_y} y_i \left(\hat{y}_i - \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right) \right)$

$= -\sum_{i=1}^{n_y} y_i \tilde{y}_i + \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right)$ (because $\sum_{i=1}^{n_y} y_i = 1$)

15. $\frac{\partial l}{\partial \tilde{y}_i} = -y_i + \frac{1}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}} e^{\tilde{y}_i} = -y_i + \hat{y}_i$

So: $\nabla_{\tilde{y}} l = \hat{y} - y$

16. • $\frac{\partial l}{\partial W^y_{ij}} = \sum_{k=1}^{n_y} \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial W^y_{ij}}$ ($\tilde{y}_i = \sum_{j=1}^{n_h} W^y_{ij} h_j + b^y_i$)

$= \sum_{k=1}^{n_y} (-y_k + \hat{y}_k) h_j \mathbb{1}_{k=i}$

$= (-y_i + \hat{y}_i) h_j$

$\nabla_{W_y} l = (\hat{y} - y)^T h$

• $\frac{\partial l}{\partial b^y_i} = \sum_{k=1}^{n_y} \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial b^y_i}$

$= \sum_{k=1}^{n_y} (-y_k + \hat{y}_k) \mathbb{1}_{k=i}$

$= \hat{y}_i - y_i$

$\nabla_{b_y} l = \hat{y} - y$

17. • $\frac{\partial l}{\partial \tilde{h}_i} = \sum_{k=1}^{n_y} \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial \tilde{h}_i} = \sum_{k=1}^{n_y} (1-h_i^2)(\hat{y}_k - y_k) W^y_{ki}$

$\frac{\partial \tilde{y}_k}{\partial \tilde{h}_i} = \sum_{j=1}^{n_h} \frac{\partial \tilde{y}_k}{\partial h_j} \frac{\partial h_j}{\partial \tilde{h}_i} = \sum_{j=1}^{n_h} W^y_{kj} (1-h_i^2) \mathbb{1}_{j=i}$ ($\frac{\partial h_i}{\partial \tilde{h}_i} = 1 - h_i^2$)

$= W^y_{ki} (1-h_i^2)$

$\nabla_{\tilde{h}} l = (\nabla_y W^y) \odot (1-h^2)$

Scanné avec CamScanner

- $\frac{\partial \ell}{\partial W^h_{ij}} = \sum_{k=1}^{n_h} \frac{\partial \ell}{\partial \tilde{h}_k} \frac{\partial \tilde{h}_k}{\partial W^h_{ij}}$ $\quad \left(\tilde{h}_i = \sum_{j=1}^{n_x} W^h_{ij} x_j + b^h_i \right)$

$$= \sum_{k=1}^{n_h} \delta^h_k x_j \mathbb{1}_{k=i}$$

$$= \delta^h_i x_j$$

$$\nabla_{W^h} = \nabla_{\tilde{h}}^T X$$

- $\frac{\partial \ell}{\partial b^h_i} = \sum_{k=1}^{n_h} \frac{\partial \ell}{\partial \tilde{h}_k} \frac{\partial \tilde{h}_k}{\partial b^h_i}$

$$= \sum_{k=1}^{n_h} \delta^h_k \mathbb{1}_{k=i}$$

$$= \delta^h_i$$

$$\nabla_{b^h} = \nabla_{\tilde{h}}^T$$

Scanné avec CamScanner

Section 2

1. After implementing the manual forward and backward passes, I experimented with different learning rates and batch sizes to observe their influence on training stability and convergence.

(1) Evolution of the loss

During training, the loss decreased steadily in the first epochs.
With an appropriate learning rate, the curve shows a fast convergence
With a too small learning rate, the curve converges slow, while a too large learning rate led to oscillations of the loss.

(2) Effect of Batch Size

Batch size	Behavior
1	noisy loss curve, fast updates but unstable accuracy
16 - 64	balanced updates, smooth convergence and good generalization
Full batch	very smooth but slow convergence per epoch, with high cost

(3) Effect of learning rate

A smaller η makes learning too slow, while a larger η causes oscillations or divergence.

Scanné avec CamScanner